

ONTOLOGIE SEMANTICĂ PENTRU BERE

Alexe Diana
Muntean Razvan

SCOPUL PROIECTULUI

.....

Dezvoltarea unei ontologii dedicate berii pentru a facilita căutarea, recomandarea și analiza tipurilor de bere, pornind de la caracteristici precum stilul, ingredientele, procesul de fabricare, conținutul de alcool și rating-urile utilizatorilor.

.....

ARCHITECTURA

.....

Date CSV

Conversie OWL

GraphDB

Interogari SPARQL

.....

DATE CSV

.....

500k randuri cu date despre bere

beer_ABV	beer_beerId	beer_brewerId	beer_name	beer_style	review_appearance	review_palette	review_overall	review_taste	review_profileName	review_aroma	review_text (scurtat)
5.0	47986	10325	Sausa Weizen	Hefeweizen	2.5	2	1.5	1.5	stcules	1.5	A lot of foam... lactic
6.2	48213	10325	Red Moon	English Strong Ale	3.0	2.5	3.0	3.0	stcules	3.0	Dark red color... cara bitterness.
6.5	48215	10325	Black Horse Black Beer	Foreign / Export Stout	3.0	2.5	3.0	3.0	stcules	3.0	Almost totally black
5.0	47969	10325	Sausa Pils	German Pilsener	3.5	3	3.0	2.5	stcules	3.0	Golden yellow... gras
7.7	64883	1075	Cauldron DIPA	American Double / Imperial IPA	4.0	4.5	4.0	4.0	johnmichaelsen	4.5	Orange copper color hoppy/bitter... very i

OWL



Structura OWL

Popularizarea ei

```
def create_ontology_structure(output_path): 1usage new *
    g = Graph()
    EX = Namespace("http://example.org/beer#")
    g.bind(prefix="ex", EX)

    # Definim clase
    for cls in ["Beer", "Brewery", "Review", "User"]:
        g.add((EX[cls], RDF.type, OWL.Class))

    # Proprietăți de obiect
    object_props = {
        "hasReview": ("Beer", "Review"),
        "hasReviewer": ("Review", "User"),
    }

    for prop, (domain, range_) in object_props.items():
        g.add((EX[prop], RDF.type, OWL.ObjectProperty))
        g.add((EX[prop], RDFS.domain, EX[domain]))
        g.add((EX[prop], RDFS.range, EX[range_]))

    # Proprietăți de dată
    datatype_props = [
        ("hasName", "string"),
        ("hasStyle", "string"),
        ("hasABV", "float"),
        ("hasAppearanceScore", "float"),
        ("hasAromaScore", "float"),
        ("hasTasteScore", "float"),
        ("hasOverallScore", "float"),
        ("hasPaletteScore", "float"),
        ("hasText", "string"),
        ("hasTime", "integer"),
    ]
```

```
# == Beer ==
beer_uri = EX[f"Beer_{row['beer_beerId']}"]
g.add((beer_uri, RDF.type, EX.Beer))
g.add((beer_uri, EX.hasName, Literal(row['beer_name'], datatype=XSD.string)))
g.add((beer_uri, EX.hasStyle, Literal(row['beer_style'], datatype=XSD.string)))
if row['beer_ABV'].strip():
    g.add((beer_uri, EX.hasABV, Literal(float(row['beer_ABV']), datatype=XSD.float)))

# == Brewery ==
brewery_uri = EX[f"Brewery_{row['beer_brewerId']}"]
g.add((brewery_uri, RDF.type, EX.Brewery))
g.add((beer_uri, EX.hasBrewery, brewery_uri))

# == Review ==
review_uri = EX[f"Review_{idx}"]
g.add((review_uri, RDF.type, EX.Review))
g.add((review_uri, EX.hasAppearanceScore, Literal(float(row['review_appearance'], datatype=XSD.float))))
g.add((review_uri, EX.hasAromaScore, Literal(float(row['review_aroma'], datatype=XSD.float))))
g.add((review_uri, EX.hasTasteScore, Literal(float(row['review_taste'], datatype=XSD.float))))
g.add((review_uri, EX.hasOverallScore, Literal(float(row['review_overall'], datatype=XSD.float))))
g.add((review_uri, EX.hasPaletteScore, Literal(float(row['review_palette'], datatype=XSD.float))))
g.add((review_uri, EX.hasText, Literal(row['review_text'], datatype=XSD.string)))
g.add((review_uri, EX.hasTime, Literal(int(row['review_time'], datatype=XSD.integer))))
```

GRAPHDB

Clasele

.....

Relatiile



....

Class	Links	
ex:Review	106K	↔
ex:Beer	56K	→
ex:User	53K	←
ex:Brewery	3K	←

INTEROGARI SPARQL

```
PREFIX ex: <http://example.org/beer#>
SELECT DISTINCT |?beerName ?tasteScore WHERE {
  ?beer ex:hasName ?beerName ;
        ex:hasReview ?review .
  ?review ex:hasTasteScore ?tasteScore .
  FILTER(?tasteScore >= 4.0)
}
LIMIT 5
```

	beerName	◆	
1	"Founders Porter"		"4.0"^^xsd:float
2	"Founders Porter"		"4.5"^^xsd:float
3	"Founders Porter"		"5.0"^^xsd:float
4	"O.K. Beer"		"4.0"^^xsd:float
5	"O.K. Beer"		"4.5"^^xsd:float

```
PREFIX ex: <http://example.org/beer#>
SELECT ?style (AVG(?overall) AS ?avgOverall) WHERE {
  ?beer ex:hasStyle ?style ;
        ex:hasReview ?review .
  ?review ex:hasOverallScore ?overall .
}
GROUP BY ?style
ORDER BY DESC(?avgOverall)
LIMIT 5
```

	style	◆	
1	"American Double / Imperial Stout"		"4.308654"^^xsd:float
2	"Keller Bier / Zwickel Bier"		"4.203704"^^xsd:float
3	"Russian Imperial Stout"		"4.1603913"^^xsd:float
4	"Rye Beer"		"4.1594043"^^xsd:float
5	"Munich Helles Lager"		"4.086758"^^xsd:float

INTEROGARI SPARQL

```
PREFIX ex: <http://example.org/beer#>
SELECT DISTINCT ?beerName ?abv WHERE {
  ?beer ex:hasName ?beerName ;
        ex:hasABV ?abv .
  FILTER(?abv > 0)
}
ORDER BY DESC(?abv)
LIMIT 5
```

	beerName	↕
1	"Schorschbräu Schorschbock 57%"	"57.7"^^xsd:float
2	"Schorschbräu Schorschbock 43%"	"43.0"^^xsd:float
3	"Schorschbräu Schorschbock 40%"	"39.44"^^xsd:float
4	"Schorschbräu Schorschbock 31%"	"30.86"^^xsd:float
5	"Schorschbock"	"16.0"^^xsd:float

```
PREFIX ex: <http://example.org/beer#>
SELECT DISTINCT ?beerName ?style WHERE {
  ?beer ex:hasName ?beerName ;
        ex:hasStyle ?style .
  FILTER(REGEX(?style, "american amber", "i"))
}
ORDER BY ?beerName
LIMIT 5
```

1	"1906 Reserva Especial Can"	"American Amber / Red Lager"
2	"Aguila Del Sur Rubia"	"American Amber / Red Ale"
3	"Ale"	"American Amber / Red Ale"
4	"Alley Way Amber"	"American Amber / Red Ale"
5	"Alright Already Amber"	"American Amber / Red Ale"

PROBLEME SI SOLUTII

.....

Numarul redus de date -
Dictionare OWL

Realizarea unui OWL din
seturi de date existente

CSV foarte mare - 500k
date

Incarcarea lor partiala



THANK YOU